MODULE 4 – INHERITANCE

1. Explain the concept of inheritance in programming.

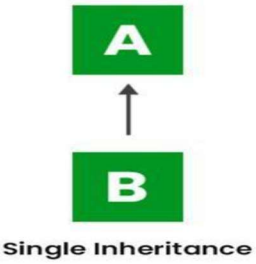
Inheritance is a concept that allows a child class to acquire the proper es and behavior of a parent class.
To inherit a class, you simply incorporate the definition of one class into another by using the extend keyword.
It promotes code reusability, method overriding, polymorphism.
Super class: the class whose features are inherited.
Sub Class: the class that inherits from another class.
Syntax:

```
Class parentclass{
//body
}
class childclass extends parentclass{
//body
}
```

Types of inheritance:

1. Single inheritance: In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class.

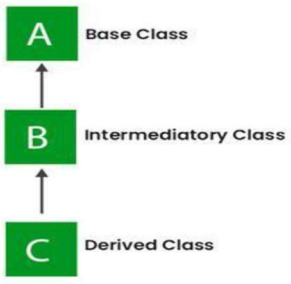
'A' is a parent class and 'B' is a child class. The class 'B' inherits all the properties of the class 'A'.



Single Inheritance

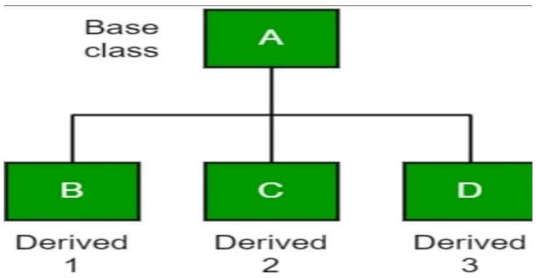
2. Multilevel inheritance: a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes.

class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C.



Multilevel Inheritance

3. Hierarchical inheritance: one class serves as a superclass (base class) for more than one subclass. class A serves as a base class for the derived classes B, C, and D.



Example Program:

```
class A {
int i;
int j;
void setij(int x, int y) {
i = x;
j = y;
}
}
class B extends A {
int total;
void sum() {
total = i + j;
}
}

class Access {
public static void main(String args[]) {
B subOb = new B();
subOb.setij(10, 12);
subOb.sum();
System.out.println("Total is " + subOb.total);
}
}
```

```
Total is 22
```

2. Illustrate multilevel hierarchy using an example.

It is a form of inheritance where a class inherits from another class, which in turn inherits from another class, crea ng a chain of inheritance.

Mul ple levels of parent-child rela onships are allowed.

Each subclass inherits proper es and methods from all its superclass in the hierarchy.

```
class GrandParent {
    void showGrandParent() {
        System.out.println("This is Grandparent class.");
    }
}
class Parent extends GrandParent {
    void showParent() {
        System.out.println("This is Parent class.");
    }
}
class Child extends Parent {
    void showChild() {
        System.out.println("This is Child class.");
    }
}
public class Multilevel {
    public static void main(String[] args) {
        Child child = new Child();
        child.showGrandParent(); // Inherited from GrandParent
        child.showParent();      // Inherited from Parent
        child.showChild();       // Child's own method
    }
}
```

```
This is Grandparent class.
This is Parent class.
This is Child class.
```

3. Demonstrate two uses of super keyword with example

1. Using super to Call Superclass Constructors

A subclass can call a constructor defined by its superclass by use of the following form of super:

super(arg-list);

Here, arg-list specifies any arguments needed by the constructor in the superclass. super( ) must always be the first statement executed inside a subclass' constructor.

To see how super( ) is used, consider this improved version of the BoxWeight( ) class:

class BoxWeight extends Box { double weight;
BoxWeight(double w, double h, double d, double m) { super(w, h, d);
weight = m;
}
}

The second form of super acts somewhat like this, except that it always refers to the superclass of the subclass in which it is used. This usage has the following general form:

super.member

Here, member can be either a method or an instance variable.

This second form of super is most applicable to situations in which member names of a subclass hide members by the same name in the superclass. Consider this simple class hierarchy:

```
class A {
int i;
}
class B extends A {
int i;
B(int a, int b) {
super.i = a;
i = b;
}
void show() {
System.out.println("i in superclass: " + super.i);
System.out.println("i in subclass: " + i);
} }
class UseSuper {
public static void main(String args[]) {
B subOb = new B(1, 2);
subOb.show();
}
}
```

```
i in superclass: 1
i in subclass: 2
```

4. Demonstrate Method Overriding using an example.

In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to *override* the method in the superclass. When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden. Consider the following:

class A {
int i, j;
A(int a, int b) {
i = a;
j = b;
}
void show() {

```
System.out.println("i and j: " + i + " " + j);
}
}
class B extends A {
int k;
B(int a, int b, int c) {
super(a, b);
k = c;
}
void show() {
System.out.println("k: " + k);
}
}
class Override {
public static void main(String args[]) {
B subOb = new B(1, 2, 3);
subOb.show();
}}
```

Output

k: 3

5. Demonstrate the working of final class and final method in JAVA.

The keyword final has three uses. First, it can be used to create the equivalent of a named constant. This use was described in the preceding chapter. The other two uses of final apply to inheritance. Both are examined here.

Using final to Prevent Overriding

While method overriding is one of Java's most powerful features, there will be times when you will want to prevent it from occurring. To disallow a method from being overridden, specify final as a modifier at the start of its declaration. Methods declared as final cannot be overridden.

```
final class A {
    void show() {
        System.out.println("Inside A");
    }
}

// class B extends A {}  // ERROR: Cannot inherit from final class A
```

Because meth( ) is declared as final, it cannot be overridden in B. If you attempt to do so, a compile-time error will result.

Methods declared as final can sometimes provide a performance enhancement: The compiler is free to inline calls to them because it "knows" they will not be overridden by a subclass. When a small final method is called, often the Java compiler can copy the bytecode for the subroutine directly inline with the compiled code of the calling method, thus eliminating the costly overhead associated with a method call. Inlining is only an option with final methods. Normally, Java resolves calls to methods dynamically, at run time. This is called late binding. However, since final methods cannot be overridden, a call to one can be resolved at compile time. This is called early binding.

Using final to Prevent Inheritance

Sometimes you will want to prevent a class from being inherited. To do this, precede the class declaration with final. Declaring a class as final implicitly declares all of its methods
as final, too. As you might expect, it is illegal to declare a class as both abstract and final since an abstract class is incomplete by itself and relies upon its subclasses to provide complete implementations.

```
class A {
final void meth() {
System.out.println("This is a final method.");
} }
class B extends A {
void meth() { // ERROR! Can't override.
System.out.println("Illegal!");
}
}
```

6. What is abstract class ? Explain with example.

An abstract class in Java is a class that cannot be instantiated on its own—it serves as a blueprint for other classes. It can include both abstract methods (without a body) and concrete methods (with a body). Abstract classes are used when you want to provide a common definition of a base class that multiple derived classes can share.

Key Features:

Declared using the abstract keyword.

Can have constructors, fields, and methods (both abstract and non-abstract).

Must be extended by a subclass to be used.

If a class has at least one abstract method, it must be declared abstract.

```
abstract class A {
abstract void callme();

void callmetoo() {
System.out.println("This is a concrete method.");
}
}
class B extends A {
void callme() {
System.out.println("B's implementation of callme.");
}
}
class AbstractDemo {
public static void main(String args[]) {
B b = new B();
b.callme();
b.callmetoo();
}
}
```

```
B's implementation of callme.
This is a concrete method.
```

7. Demonstrate dynamic dispatch using abstract class in JAVA

```
class A {
void callme() {
 System.out.println("Inside A's callme method");
}
}
class B extends A {
void callme() {
 System.out.println("Inside B's callme method");
}
}
class C extends A{
void callme() {
 System.out.println("Inside C's callme method");
}
}
class Dispatch {
public static void main(String args[]) {
 A   a = new A();
 B   b = new B();
 C   c = new C();
 A r;
 r = a;
 r.callme();
}
}
```

```
Inside A's callme method
```

8. When constructors are called in inheritance.

When a class hierarchy is created, in what order are the constructors for the classes that make up the hierarchy called? For example, given a subclass called **B** and a superclass called **A**, is **A**'s constructor called before **B**'s, or vice versa? The answer is that in a class hierarchy, constructors are called in order of derivation, from superclass to subclass. Further, since **super( )** must be the first statement executed in a subclass' constructor, this order is the same whether or not **super( )** is used. If **super( )** is not used, then the default or parameterless constructor of each superclass will be executed. The following program illustrates when constructors are executed:

```
class A {
A() {
System.out.println("Inside A's constructor.");
}
}
class B extends A {
B() {
System.out.println("Inside B's constructor.");
}
}
class C extends B {
C() {
System.out.println("Inside C's constructor.");
  } }
class CallingCons {
```

```
public static void main(String args[]) {
   C c = new C();
   }
   }
```

Output

Inside A's constructor

Inside B's constructor

Inside C's constructor

ACET